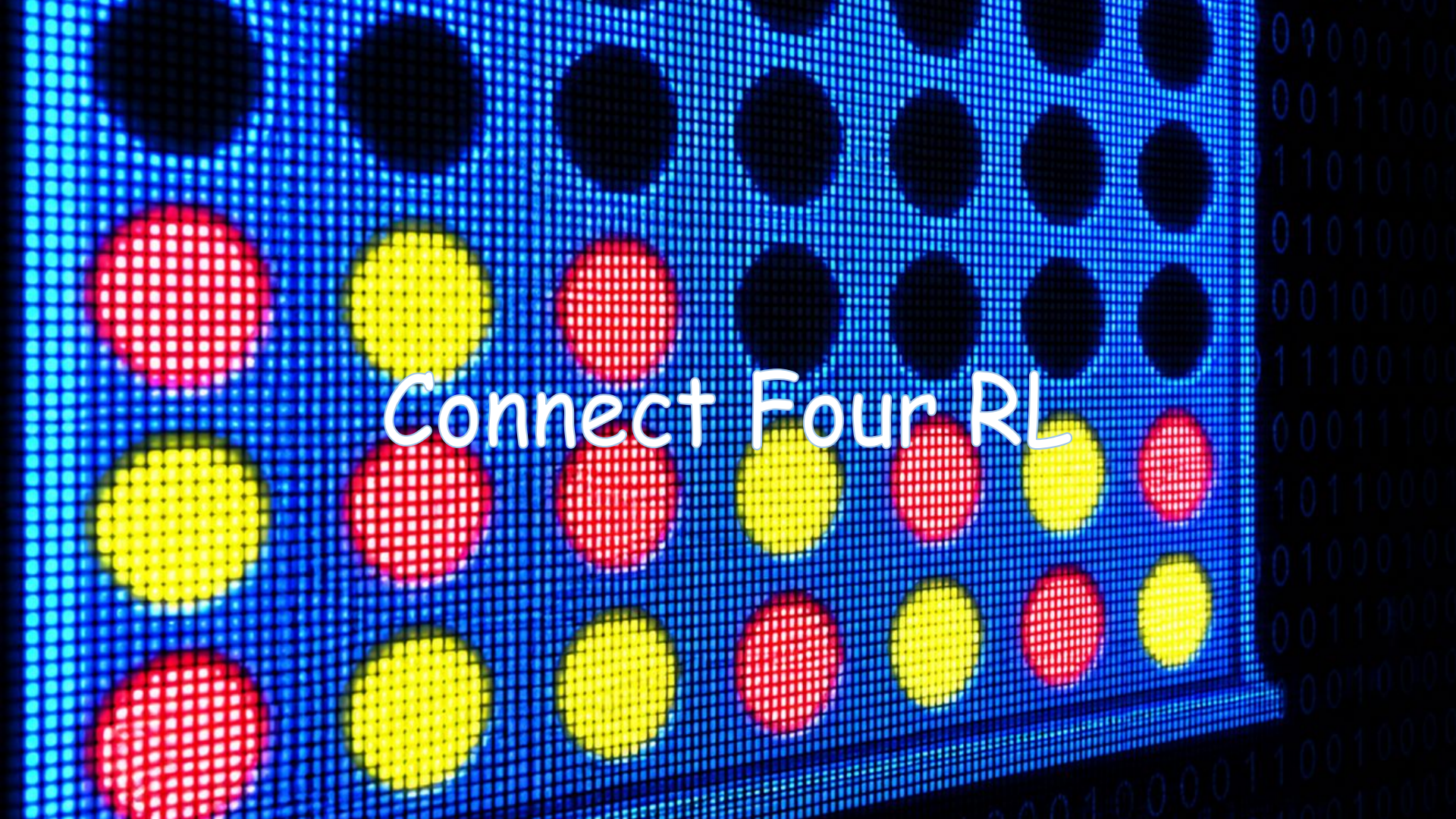


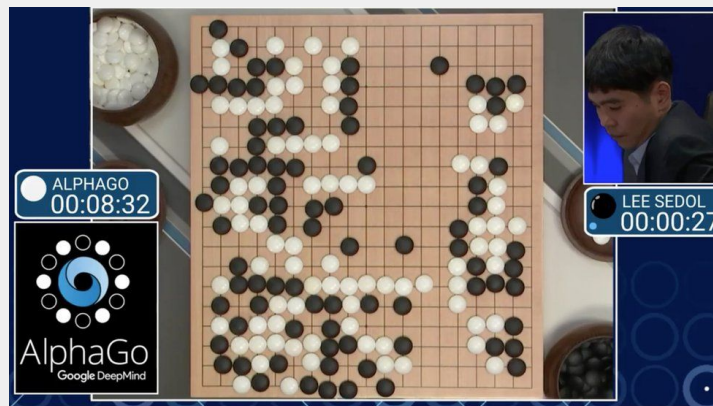


Connect Four RL

Monte Carlo Tree Search (MCTS):

- MCTS: a heuristic search algorithm used for decision making in complex, large-scale search spaces such as Go, or chess.
- Unlike minimax, MCTS does not create nor search the entire game tree. Instead, it builds a partial tree through repeated simulations on a copy of the current game state.
- It samples random playouts from this copy, repeatedly selecting the most promising moves until converging toward strong play.

Google DeepMind's AlphaGo beats 18-time world champion Lee Sedol 4-1 in a five-game match in Seoul (2016).



MCTS Step 1: Selection

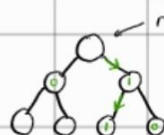
Many successive iterations of MCTS per game loop.

one iteration: (same tree grows and is shared across all iterations. Every game loop starts its own tree)

Selection

- Starting at tree's root node, move down tree using UCB1: $UCB1(i) = \bar{x}_i + c \sqrt{\frac{\ln(N)}{n_i}}$ ← balances exploitation/exploration
- Pick child with highest UCB1 score, and repeat until:
 - Unexpanded node
 - Terminal node (go straight to backpropagation)
- move onto expansion phase

current tree:



root is current game state

compute UCB1:
move down to child
with higher UCB1. Repeat
until terminal/unexpanded node.

UCB1:

exploit = child wins / child visits

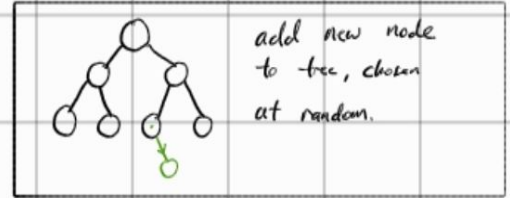
explore = $c \times \sqrt{\frac{\ln(\text{self visits})}{\text{child visits}}}$ ← $c = \sqrt{2}$, the exploration parameter

UCB = exploit + explore

MCTS Step 2: Expansion

Expansion

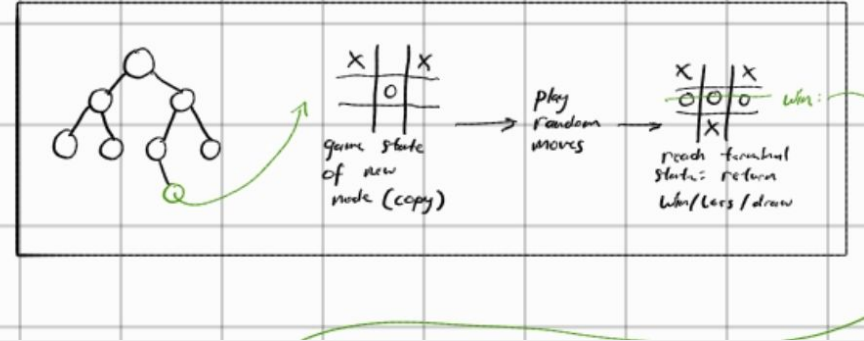
- when selection reaches a non-terminal leaf node, expand the tree by adding one or more child nodes (possible actions from current state).
- this new node is chosen at random until we use a neural net.
- if no new moves to play, go straight to backpropagation.



MCTS Step 3: Simulation

Simulation

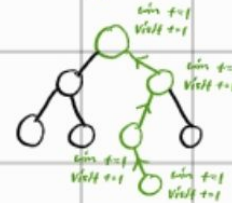
- Make a copy of the board state of new node.
- Play random moves until a terminal state:
 - win
 - loss
 - draw (full board)
- return this result to backpropagation phase.



MCTS Step 4: Backpropagation

Backpropagation

- go from newly expanded node back up to the root, updating data:
 - Visits += 1
 - Wins += 1 if simulation was a win, else wins += 0



- from new node, propagate up until the root node.
 - Visits += 1
 - Wins += 1 if win, else Wins += 0

Repeat

Process

1. game logic

```
transpose: proper game board display
tensor([[ 0,  0,  0,  0,  0,  0,  0],
        [ 0,  0,  0,  0,  0,  0,  0],
        [ 0,  0,  0,  1,  0,  0,  0],
        [ 0,  0,  1,  1,  0,  0,  0],
        [-1,  1, -1, -1,  0,  0,  0],
        [ 1, -1,  1, -1,  1,  0, -1]], dtype=torch.int8)
all possible moves left: [[0, 3], [1, 3], [2, 2], [3, 1], [4, 4], [5, 5], [6, 4]]
player 1 just played [3, 2]
-----
four in a row: True
PS C:\Users\25jov\Desktop\Connect Four ML>
```

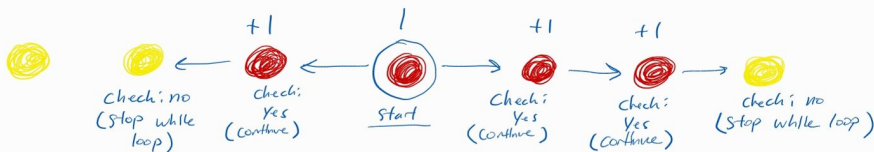
2. playable game against yourself (in terminal)

```
tensor([[ 0,  0,  0,  0,  0,  0,  0],
        [ 0,  0,  0,  0,  0,  0,  0],
        [ 0,  0,  0,  0,  0,  0,  0],
        [ 0,  0,  1,  0,  0,  0,  0],
        [ 0,  1,  1,  0,  0,  0,  0],
        [ 1, -1, -1, -1,  0,  0,  0]], dtype=torch.int8)
player -1 turn: insert into column with space (0-6): 5
four in a row: False
-----
tensor([[ 0,  0,  0,  0,  0,  0,  0],
        [ 0,  0,  0,  0,  0,  0,  0],
        [ 0,  0,  0,  0,  0,  0,  0],
        [ 0,  0,  1,  0,  0,  0,  0],
        [ 0,  1,  1,  0,  0,  0,  0],
        [ 1, -1, -1, -1,  0, -1,  0]], dtype=torch.int8)
player 1 turn: insert into column with space (0-6): 3
four in a row: False
-----
tensor([[ 0,  0,  0,  0,  0,  0,  0],
        [ 0,  0,  0,  0,  0,  0,  0],
        [ 0,  0,  0,  0,  0,  0,  0],
        [ 0,  0,  1,  0,  0,  0,  0],
        [ 0,  1,  1,  1,  0,  0,  0],
        [ 1, -1, -1, -1,  0, -1,  0]], dtype=torch.int8)
player -1 turn: insert into column with space (0-6): 3
four in a row: False
-----
tensor([[ 0,  0,  0,  0,  0,  0,  0],
        [ 0,  0,  0,  0,  0,  0,  0],
        [ 0,  0,  0,  0,  0,  0,  0],
        [ 0,  0,  1, -1,  0,  0,  0],
        [ 0,  1,  1,  1,  0,  0,  0],
        [ 1, -1, -1, -1,  0, -1,  0]], dtype=torch.int8)
player 1 turn: insert into column with space (0-6): 3
four in a row: True
-----
FOUR IN A ROW!
tensor([[ 0,  0,  0,  0,  0,  0,  0],
        [ 0,  0,  0,  0,  0,  0,  0],
        [ 0,  0,  0,  1,  0,  0,  0],
        [ 0,  0,  1, -1,  0,  0,  0],
        [ 0,  1,  1,  1,  0,  0,  0],
        [ 1, -1, -1, -1,  0, -1,  0]], dtype=torch.int8)
WINNER: 1
PS C:\Users\25jov\Desktop\Connect Four ML>
```

Process

3. MCTS implemented

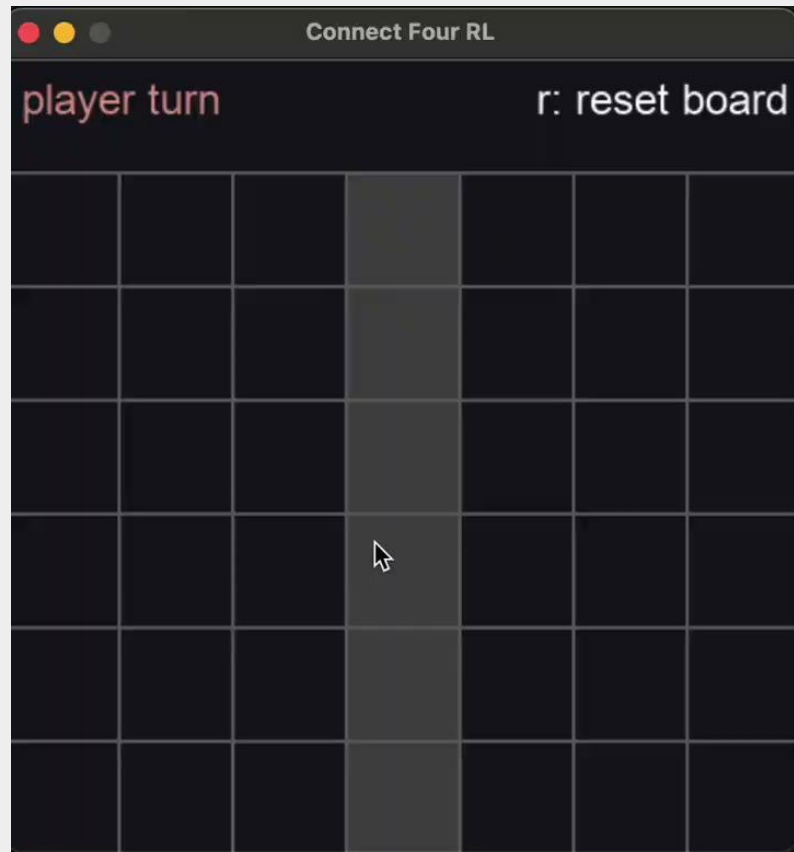
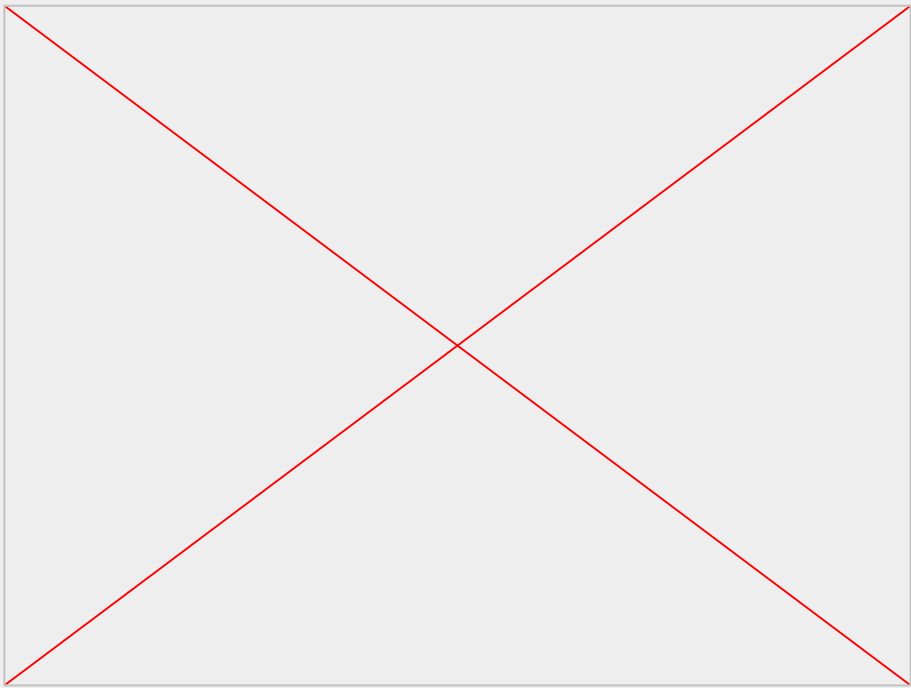
```
-----  
tensor([[ 0,  0,  0,  0,  0,  0,  0],  
        [ 0,  0,  0,  0,  0,  0,  0],  
        [ 0,  0,  0,  0,  0, -1,  0],  
        [ 0,  0,  0, -1, -1,  1,  0],  
        [ 0,  0,  0,  1, -1,  1,  0],  
        [ 0,  0, -1,  1,  1,  1, -1]], dtype=torch.int8)  
player 1 turn: insert into column with space (0-6):
```



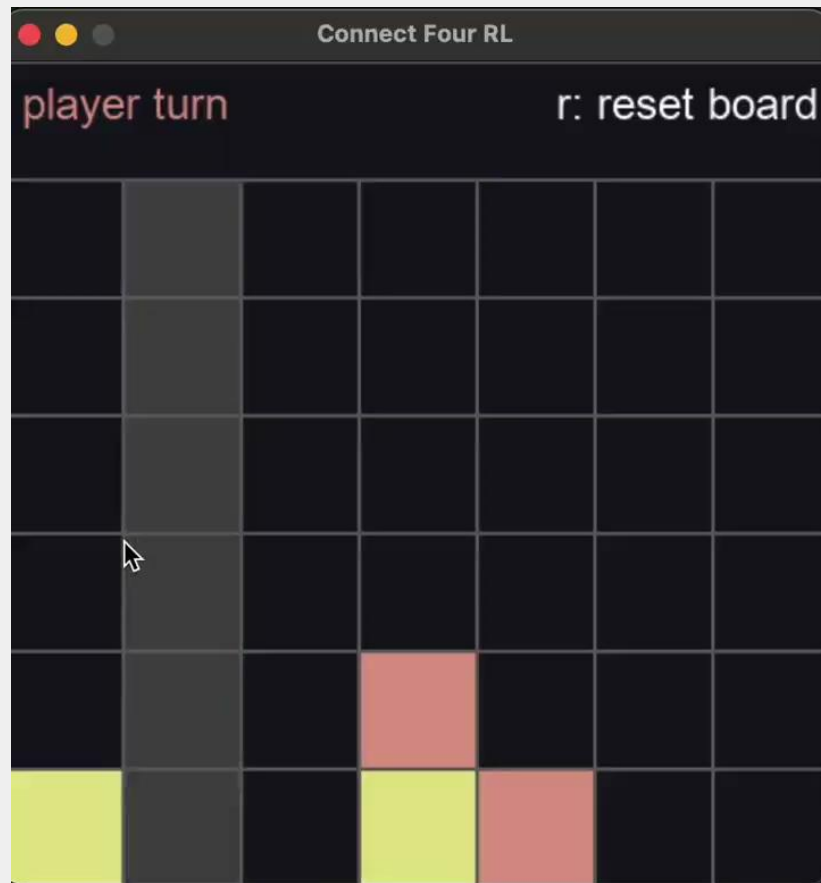
4. One-move heuristics

```
# heuristics  
# easy win: get_current_player()  
# easy block: get_last_player()  
  
def one_move_heuristic(self, check):  
    check_moves = self.get_all_possible_moves(self.board)  
    for move in check_moves:  
        dummy_board = deepcopy(self.board)  
        col = move[0] # get col  
        coords = self.find_next_in_col(dummy_board, col)  
        dummy_board[coords[0]][coords[1]] = check  
        if (self.check_win_state(dummy_board, coords) == True):  
            self.place_move(self.board, -1, col)  
            return True  
    return False  
  
# if there is a win in 1, make that move.  
def easy_win_heuristic(self):  
    return self.one_move_heuristic(self.get_current_player())  
  
# if opponent has win in 1, block that move.  
def easy_block_heuristic(self):  
    return self.one_move_heuristic(self.get_last_player())
```

Video Demonstration: Machine Win



Video Demonstration: Player Win



Thank you!

